ORACLE

Taming Zombie Cgroups: Should Reparenting Be Simple or Smart?

Harry Yoo, Kamalesh Babulal

harry.yoo@oracle.com, kamalesh.babulal@oracle.com

Oracle Cloud Infrastructure

December 11, 2025

Safe harbor statement

The following is intended to outline our general product direction. It is intended for information purposes only, and may not be incorporated into any contract. It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions. The development, release, timing, and pricing of any features or functionality described for Oracle's products may change and remains at the sole discretion of Oracle Corporation.

Copyright © 2025, Oracle and/or its affiliates |

Quick Recap: Zombie Memory Cgroup Problem

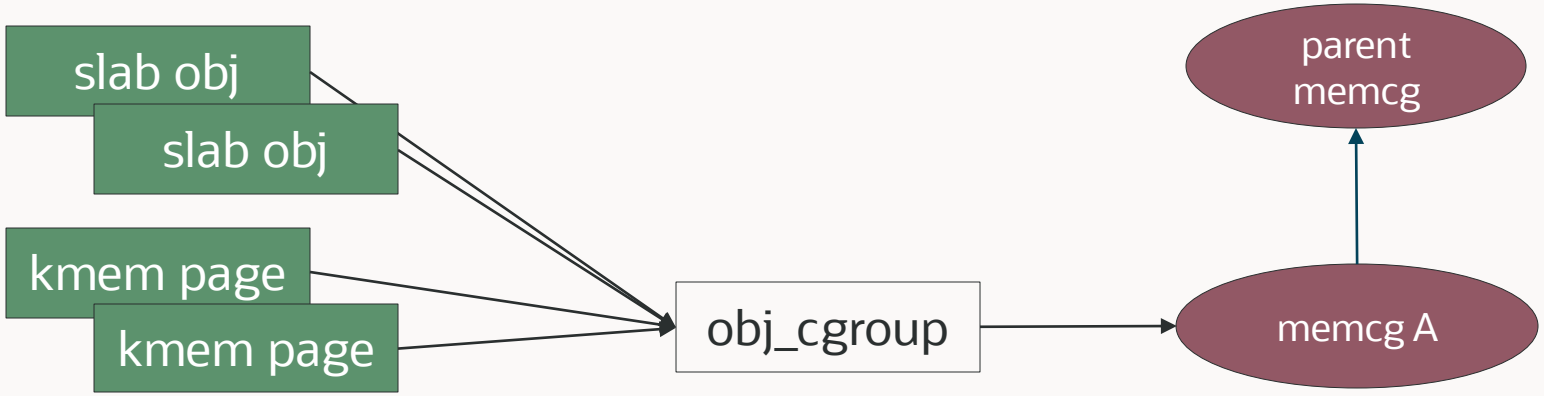
Zombie Memcg Problem: memory cgroups are removed from the hierarchy, but they cannot disappear due to pages outlive them

- When pages are shared between memory cgroups, they could outlive the memory cgroup.
- Because each LRU page take a reference to its memory cgroup, it cannot disappear until all (shared) are reclaimed (become zombie).
- As zombie memory cgroups accumulate, the system could waste a huge amount of memory (mainly due to struct mem_cgroup and percpu data)

Copyright © 2025, Oracle and/or its affiliates |

Quick Recap: obj_cgroup API

- Kernel pages are charged via obj_cgroup API and they are reparented to the parent when a memcg is offlined (thanks to Roman Guschin)
- Let's eliminate the zombie memory cgroup problem: charge LRU folios with objcg, and reparent them on memcg offlining!
- The latest patch series posted on Oct 2025, by Qi Zheng & Muchun Song @ ByteDance



Copyright © 2025, Oracle and/or its affiliates | Linux Plumbers Conference, Tokyo 2025

The main agenda today

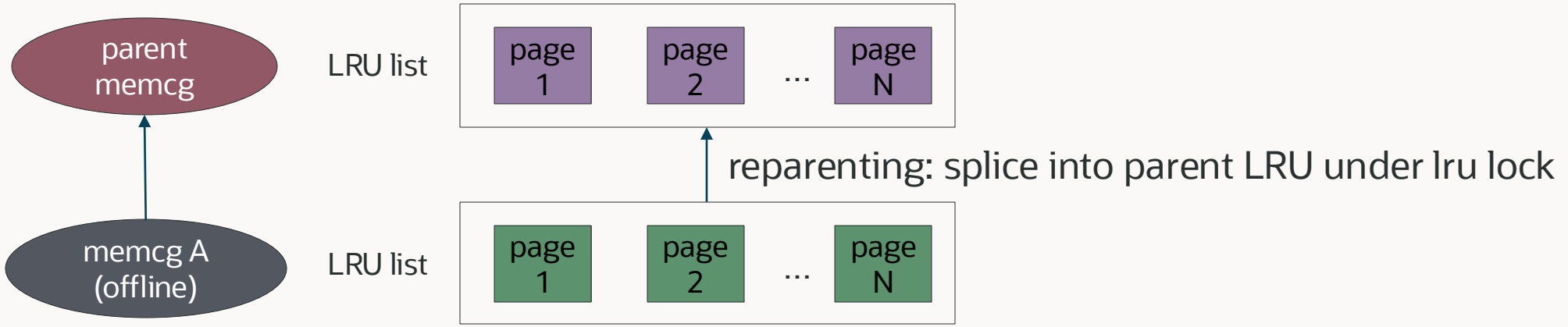
Reparenting can be implemented in either a simple or smart (complicated) method.

- The simple method doesn't respect hotness of pages being reparented
  - Probably not optimal and may cause LRU disorder
  - But it's simple and it works
- The smart (*complicated*) method may perform better, but it's quite complicated

The question: any real-world workloads that'll justify the complexity of the smart method?

Copyright © 2025, Oracle and/or its affiliates |

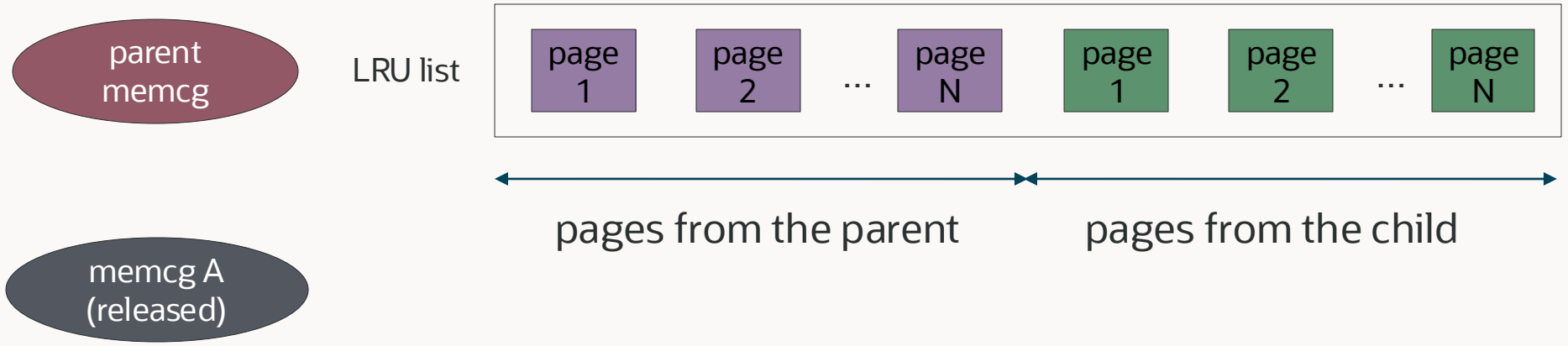
“Simple” reparenting method: splice LRU list into the parent’s



- “Simple” reparenting: splice child LRU list into parent’s, upon memcg offlining
- memcg A can be released after reparenting

Copyright © 2025, Oracle and/or its affiliates |

Merging two LRU lists can cause LRU disorder



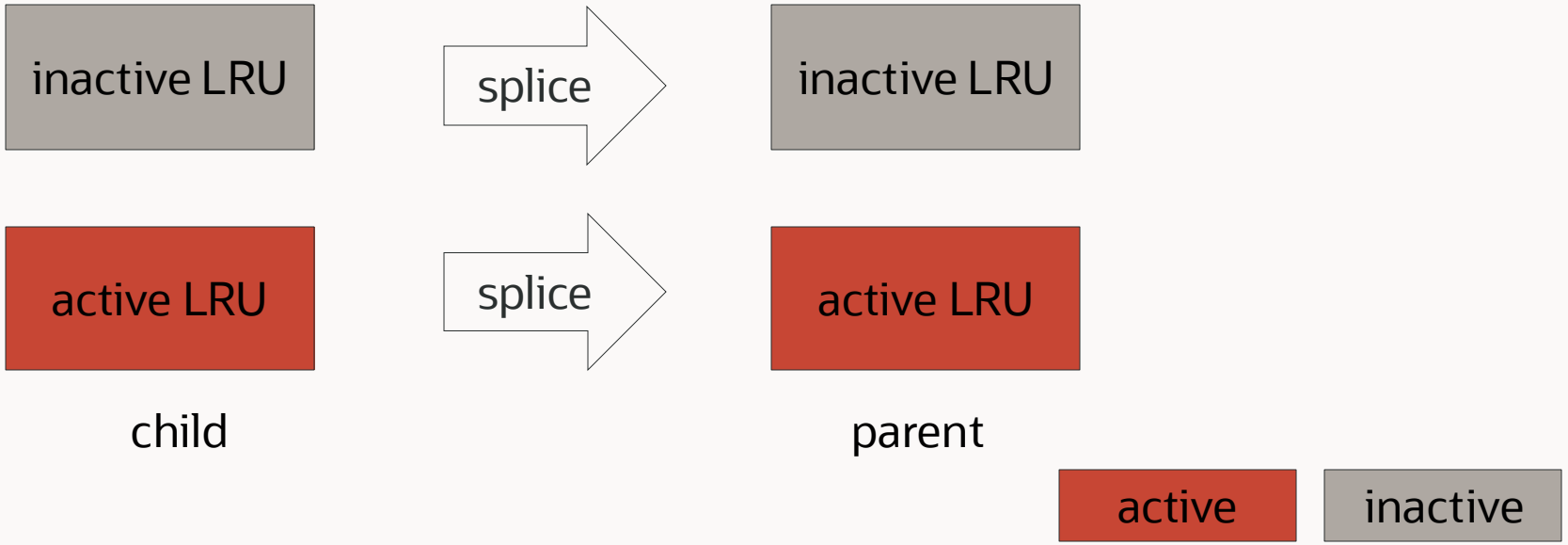
- Pages from the parent are suddenly considered “colder” compared to the child
- Basically we’re adding noise to the LRU mechanism upon reparenting
- Reclaim can be inefficient when reclaiming wrong pages due to reparenting

Copyright © 2025, Oracle and/or its affiliates | Linux Plumbers Conference, Tokyo 2025

“Simple” reparenting method (w/ traditional LRU)

Traditional LRU

- Acquire child and parent’s LRU lock
- Splice LRU lists to matching LRU list in parent
- This does not respect hotness of pages, but it’s simple!

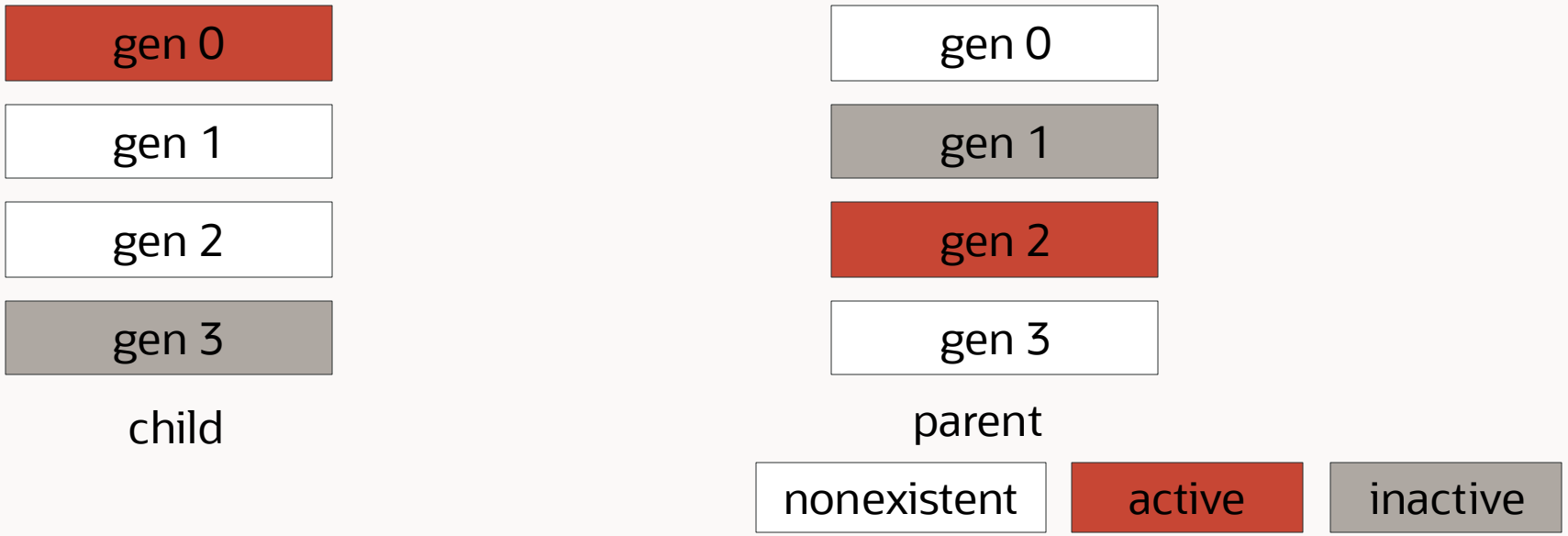


Copyright © 2025, Oracle and/or its affiliates |

“Simple” reparenting method (w/ MGLRU)

Multi-generational LRU

- Acquire child and parent’s LRU lock

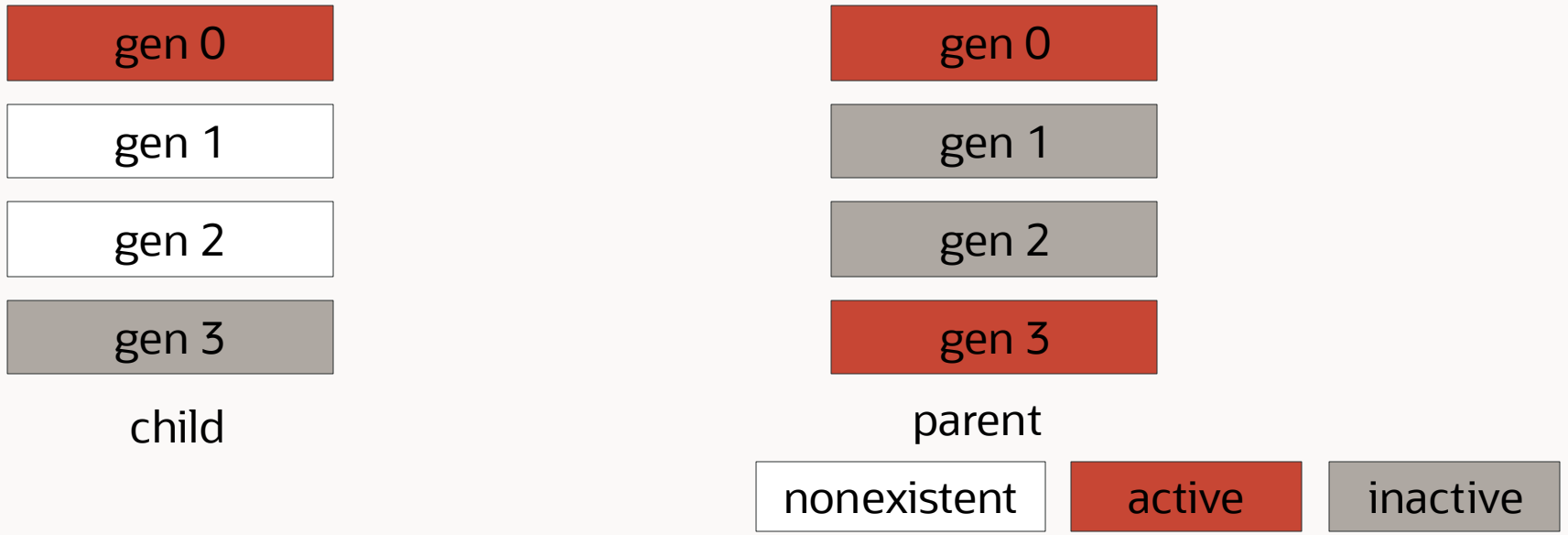


Copyright © 2025, Oracle and/or its affiliates |

“Simple” reparenting method (w/ MGLRU)

Multi-generational LRU

- Acquire child and parent’s LRU lock
- Max out the parent generations to MAX_NR_GENS

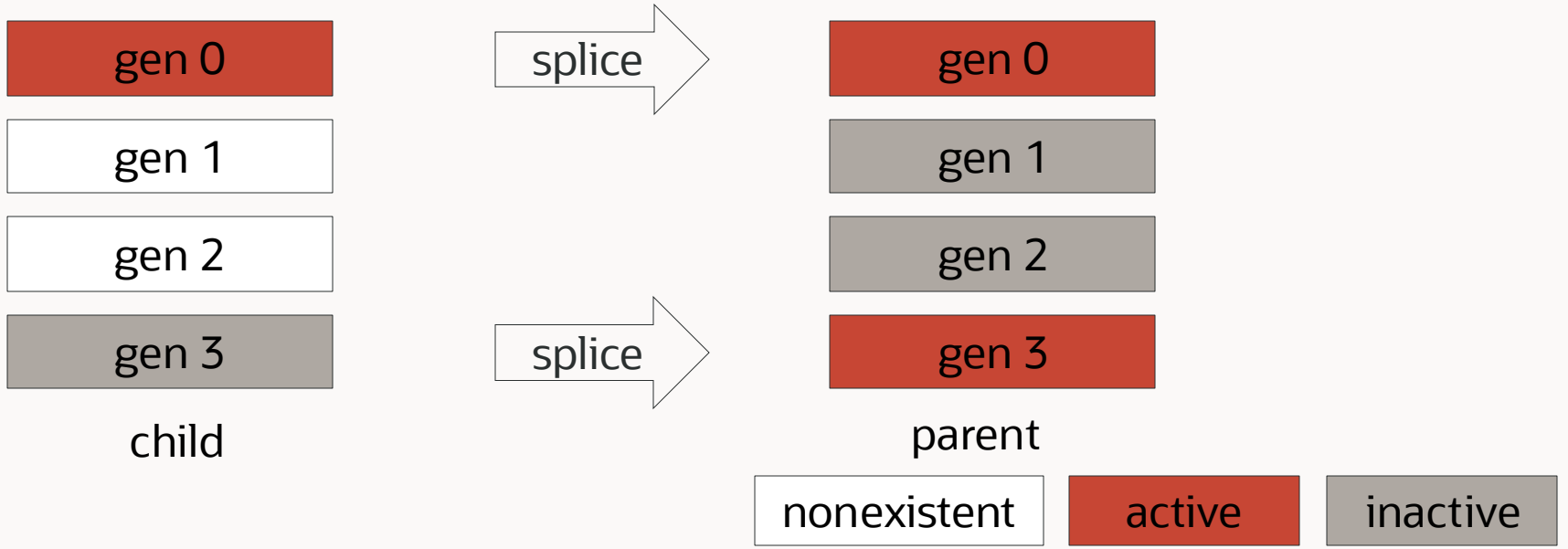


Copyright © 2025, Oracle and/or its affiliates |

“Simple” reparenting method (w/ MGLRU)

Multi-generational LRU

- Acquire child and parent’s LRU lock
- Max out the parent generations to MAX_NR_GENS
- Splice existing child generations to matching ones in parent



Copyright © 2025, Oracle and/or its affiliates |

Challenges of “smart” reparenting method (1)

”Smart” method comes with a few challenges:

- It is challenging to merge two LRU lists “optimally”
  - We don’t know how hot they are compared to pages in other LRU lists
  - e.g.) the coldest page in parent might be hotter than the hottest in child!

Copyright © 2025, Oracle and/or its affiliates |

Challenges of “smart” reparenting method (2)

”Smart” method comes with a few challenges:

- We have to iterate over all folios to make better decisions
  - We can’t iterate all folios under the LRU lock (otherwise soft lockup)
  - We can’t release lruvec lock in the midway (or folios might be isolated during reparenting)

- Thus, “smart” version needs to implement “deferred reparenting”
  - Isolate pages and process them after reparenting
    - pages cannot be reclaimed until being processed
  - Need to consider issues with concurrent aging, compaction, migration, etc.

Copyright © 2025, Oracle and/or its affiliates |

Backup Slides

A theoretical adversarial workload

Memory manager may keep reclaiming wrong pages in such a scenario:

- Create a cgroup where its processes touch a huge amount of files in a short period
- Delete the cgroup after the processes exit
- Repeat this over and over

Does this sound realistic enough to consider more sophiscated method?

Copyright © 2025, Oracle and/or its affiliates |

Recharging vs. Reparenting

Recharging: charge the pages to other memory cgroups

- Tricky because we don't know what other memcgs are accessing those pages
- Attempt by Yosry Ahmed @ Google in 2023 [1], but rejected by Johannes [2] because:
  - It is not consistent to apply reparenting for kernel memory and recharging for user memory
  - Unlike reparenting, recharging changes user-facing semantics: since resources belonging to a cgroup are charged to its all ancestors as well, it is changing the semantics to charge it to another cgroup other than its parent

[1] https://lore.kernel.org/lkml/20230720070825.992023-1-yosryahmed@google.com
[2] https://lore.kernel.org/lkml/20230720153515.GA1003248@cmpxchg.org

Copyright © 2025, Oracle and/or its affiliates |

Recharging vs. Reparenting

Reparenting: transfer pages from the child to the parent

- Not a perfect solution, but the most viable one so far
- Does not change user-facing semantics
- Reparenting fixes zombie memcg problem, but not the problem with incorrect accounting of shared resources
  - But the other problem is challenging to fix without introducing new problems

Copyright © 2025, Oracle and/or its affiliates |

Reparenting: LRU pages no longer pin its memory cgroup!

LRU pages are charged via objcg API and they pin objcg instead of memcg

- folio_memcg() & folio_lruvec() users needs to if they want stable folio<->memcg binding
- folio_memcg() & folio_lruvec() users have to acquire RCU read lock
- folio<->memcg binding becomes stable only after acquiring LRU lock

```
struct lruvec *folio_lruvec_lock(struct folio *folio)
{
        struct lruvec *lruvec = folio_lruvec(folio);

        spin_lock(&lruvec->lru_lock);
        lruvec_memcg_debug(lruvec, folio);

        return lruvec;
}
```

w/o reparenting

```
struct lruvec *folio_lruvec_lock(struct folio *folio)
{
        struct lruvec *lruvec;

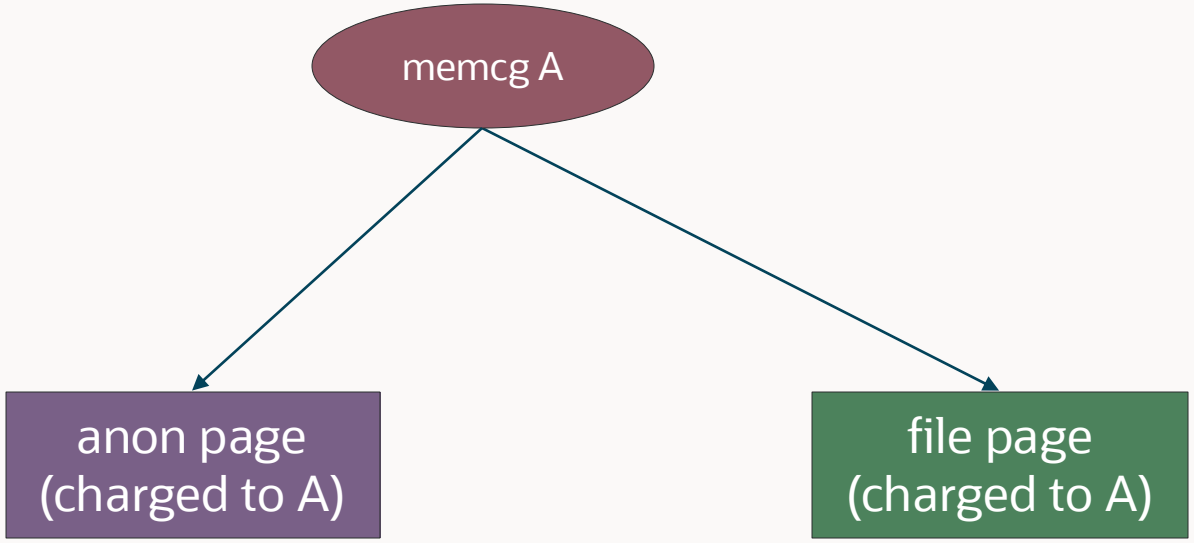
        rcu_read_lock();
retry:
        lruvec = folio_lruvec(folio);
        spin_lock(&lruvec->lru_lock);
        if (unlikely(lruvec_memcg(lruvec) != folio_memcg(folio))) {
                spin_unlock(&lruvec->lru_lock);
                goto retry;
        }

        return lruvec;
}
```

w/ reparenting

Copyright © 2025, Oracle and/or its affiliates |

Memory cgroups track how much memory is allocated for cgroup

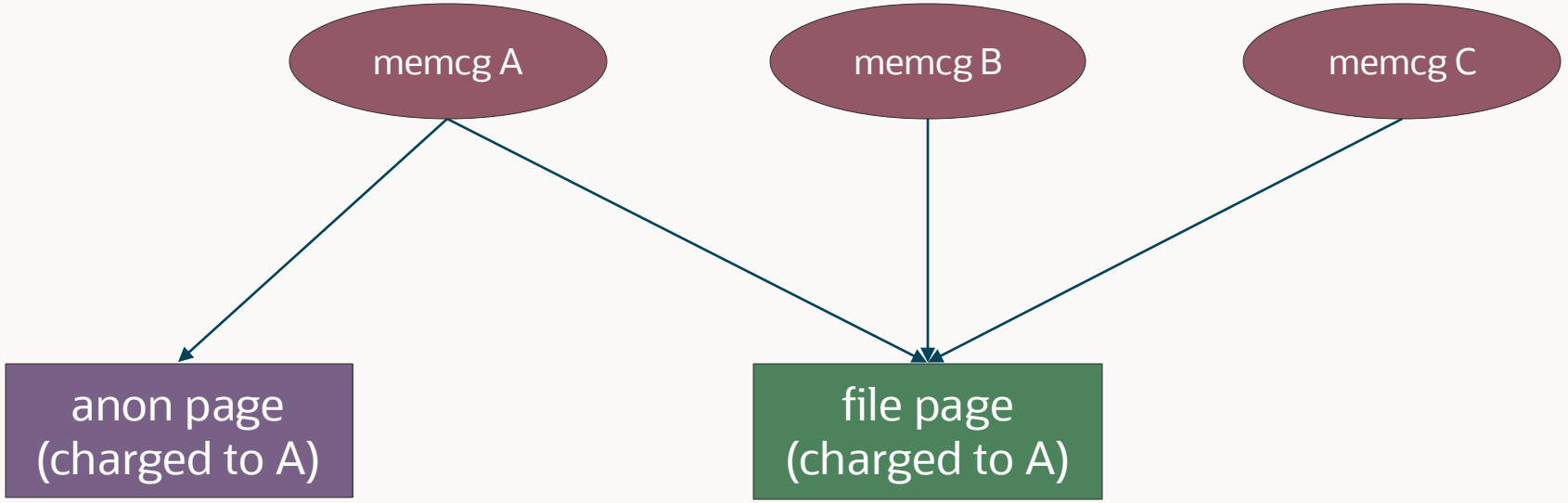


- memcg A allocates a file page (might be shared to other memcgs)
- memcg A allocates an anon page (private to memcg A)
- memcg A gets charged for both of them

Copyright © 2025, Oracle and/or its affiliates |

Shared pages are duly charged – only the first memcg

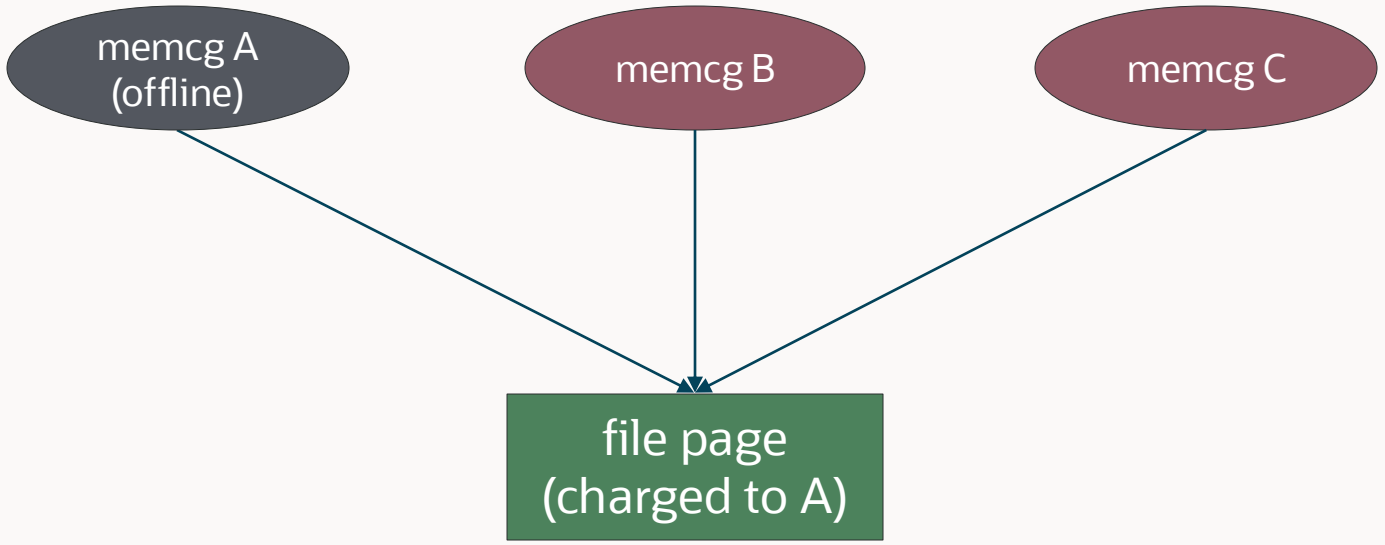
Memory controller is never designed to work well with shared resources



- memcg B, C accesses the file page, but don't get charged and access it for free
- When pages are shared, only the first memcg gets charged for it!

Copyright © 2025, Oracle and/or its affiliates |

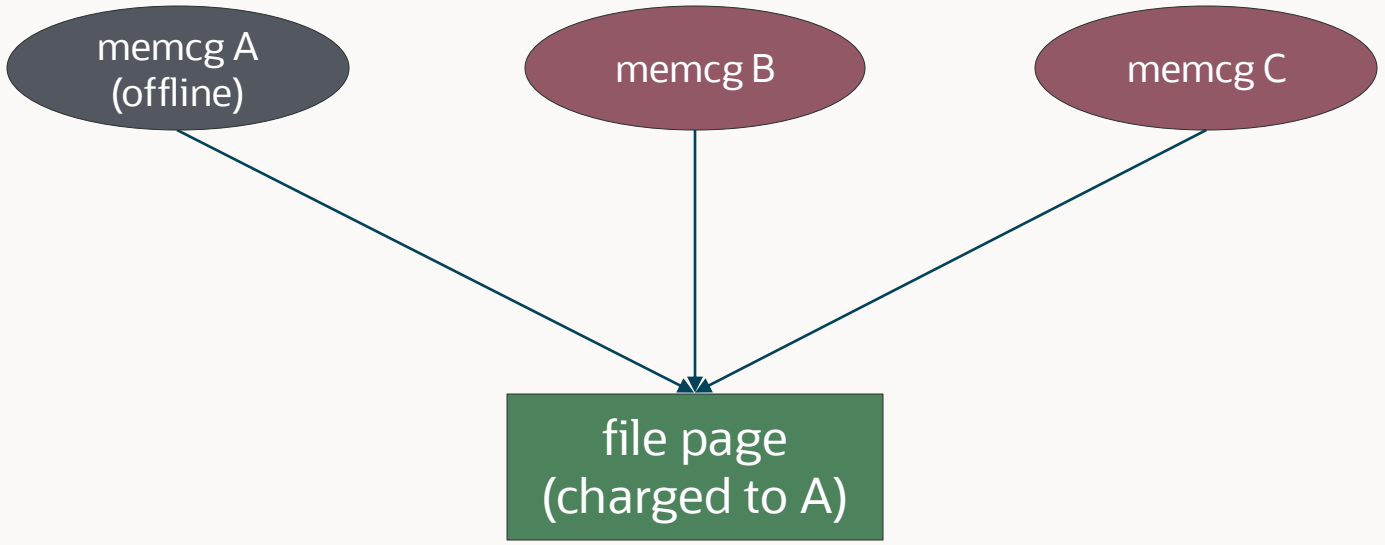
What happens when memcg A gets offlined?



- memcg A is removed from the hierarchy, but the file page is still charged to A
- memcg A cannot be released until the file page is reclaimed, **becomes a zombie**
- Unlike the file page, the anon page is reclaimed before offlining because it is not shared

Copyright © 2025, Oracle and/or its affiliates |

Zombies accumulate, memory gets wasted



Zombie Memcg Problem

- Memory cgroups becomes zombie due to their shared resources outlive memcg themselves
- As cgroups are created and destroyed, zombie memcgs accumulate and waste memory

Copyright © 2025, Oracle and/or its affiliates | Linux Plumbers Conference, Tokyo 2025


ORACLE